

```
for epoch in range(epochs):
    # Forward pass
    z1 = X @ W1 + b1
    a1 = torch.sigmoid(z1)

    z2 = a1 @ W2 + b2
    a2 = torch.sigmoid(z2)

    z3 = a2 @ W3 + b3
    y_pred = torch.sigmoid(z3)

    loss = F.binary_cross_entropy(y_pred, y)
    loss.backward()
    with torch.no_grad():
        W1 -= lr * W1.grad
        b1 -= lr * b1.grad

        W2 -= lr * W2.grad
        b2 -= lr * b2.grad

        W3 -= lr * W3.grad
        b3 -= lr * b3.grad

    # Zero the gradients
    W1.grad.zero_()
    b1.grad.zero_()
    W2.grad.zero_()
    b2.grad.zero_()
    W3.grad.zero_()
    b3.grad.zero_()

    loss_history.append(loss.item())
```

```
if epoch % 500 == 0:  
    print(f"Epoch {epoch} - Loss: {loss.item():.4f}")..
```

```
Epoch 0 - Loss: 0.7198  
Epoch 500 - Loss: 0.6869  
Epoch 1000 - Loss: 0.6755  
Epoch 1500 - Loss: 0.6402  
Epoch 2000 - Loss: 0.5343  
Epoch 2500 - Loss: 0.2707  
Epoch 3000 - Loss: 0.0889  
Epoch 3500 - Loss: 0.0417  
Epoch 4000 - Loss: 0.0253  
Epoch 4500 - Loss: 0.0177
```

```
plt.plot(loss_history)  
plt.title("Loss over Epochs")  
plt.xlabel("Epoch")  
plt.ylabel("Binary Cross Entropy Loss")  
plt.grid(True)  
plt.show()
```

```
with torch.no_grad():
    z1 = X @ W1 + b1
    a1 = torch.sigmoid(z1)

    z2 = a1 @ W2 + b2
    a2 = torch.sigmoid(z2)

    z3 = a2 @ W3 + b3
    y_pred = torch.sigmoid(z3)
    predicted = (y_pred > 0.5).float()

print("Predictions:\n", predicted)
print("Ground Truth:\n", y)
```

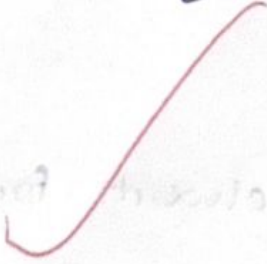
```
Predictions:
tensor([[0.],
        [1.],
        [1.],
        [0.]])
Ground Truth:
tensor([[0.],
        [1.],
        [1.],
        [0.]])
```

predictions:-

tensor ([[0.],
[1.],
[1.],
[0.]])

Ground Truth:

tensor ([[0.],
[1.],
[1.],
[0.]])



- $A_1 = \text{activation}(z_1)$ # sigmoid or ReLU
- $\hat{A}_2 = w_2 + A_1 + b_2$
- $A_2 = \text{activation}(z_2)$ # sigmoid (softmax output).

4. compute loss using mean squared error or cross entropy
5. backpropagation.

- compute error at output ($dA_2 = A_2 - y$)
- calculate gradients for w_2, b_2
- propagate error to hidden layer
- calculate gradient for w_1, b_1

6. update weight and biases using gradient descent:

$$w = w - \eta + dw$$

$$b = b - \eta + db$$

7. Repeat the step 3-6 for given number of epoch
8. Observe loss reduction and accuracy improvement
END.

Observation:-

- loss decrease as the number of iteration (epoch) increase
- weights and biases adjust to minimize error.
- Backpropagation, ensures errors are efficiently distributed layer by layer.
- Learning rate (η) greatly influences convergence speed.

Result:-

~~Successfully~~ Successfully implemented gradient descent and backpropagation in deep neural network.

6. Implement gradient descent & backpropagation in deep neural network.

Aim:-

To implement gradient descent and backpropagation algorithm in a simple deep neural network and study their role in training.

Objectives:-

1. To understand the working of gradient descent as an optimization method.
2. To implement backpropagation for updating weights in neural network.
3. To train a simple neural network for a classification task using the algorithm.
4. To observe how loss decrease with iteration.

Pseudo code:-

Start

1. Initiate dataset (x, y) for training (eg: XOR dataset)
2. Initialize neural network parameters:
 - Input layers, hidden layer, output layer size.
 - Random weights & biases.
 - Learning rate (η) .
3. Define forward propagation:
 - $z_1 = w_1 * x + b_1$

epoch 0 - loss : 0.7198

epoch 500 - loss : 0.6869

epoch 1000 - loss : 0.6455

epoch 1500 - loss : 0.6402

epoch ~~1800~~ - loss : ~~0.5343~~

epoch 2000 - loss : 0.5343

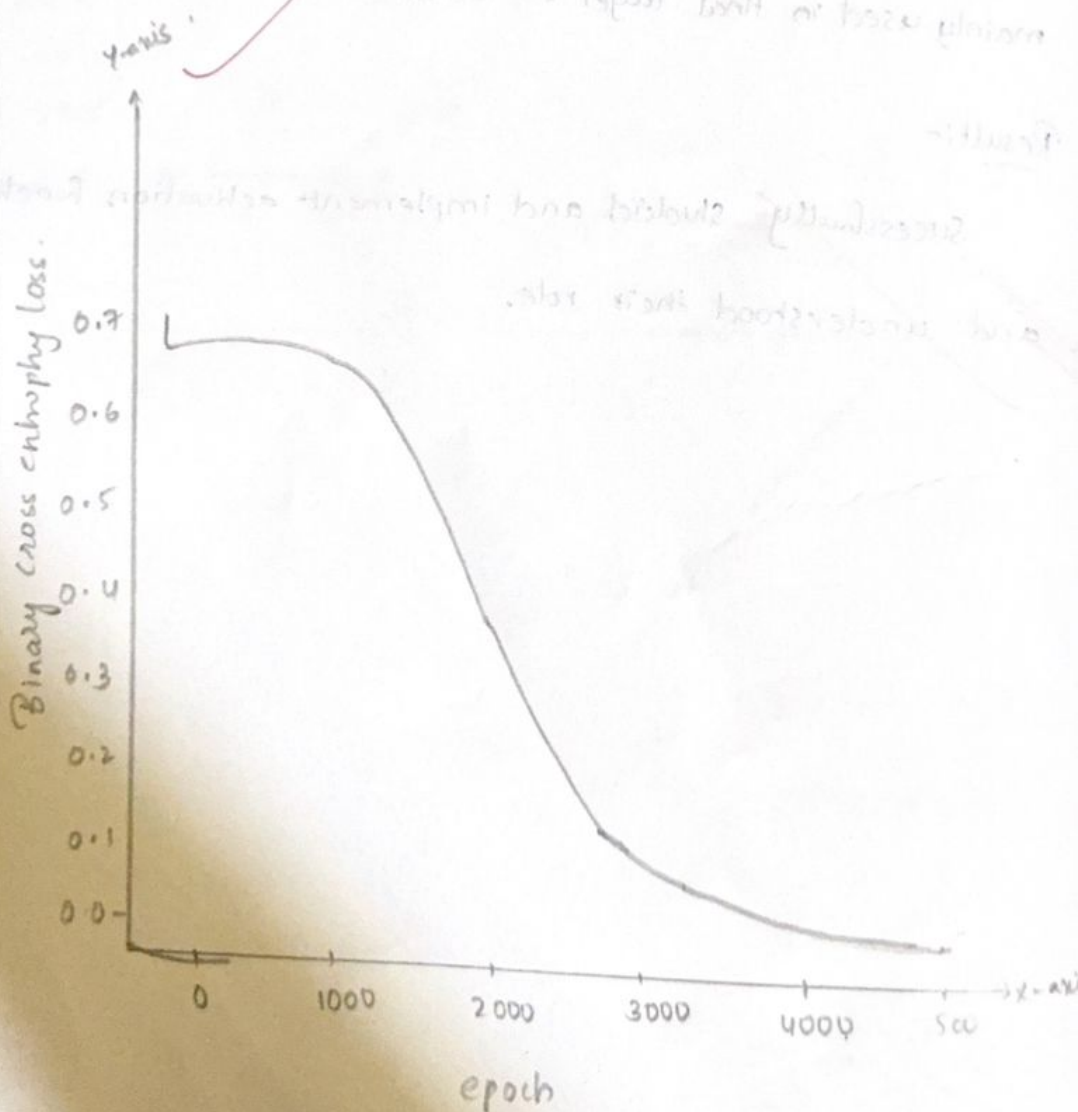
epoch 2500 - loss : 0.2404

epoch 3000 - loss : 0.0889

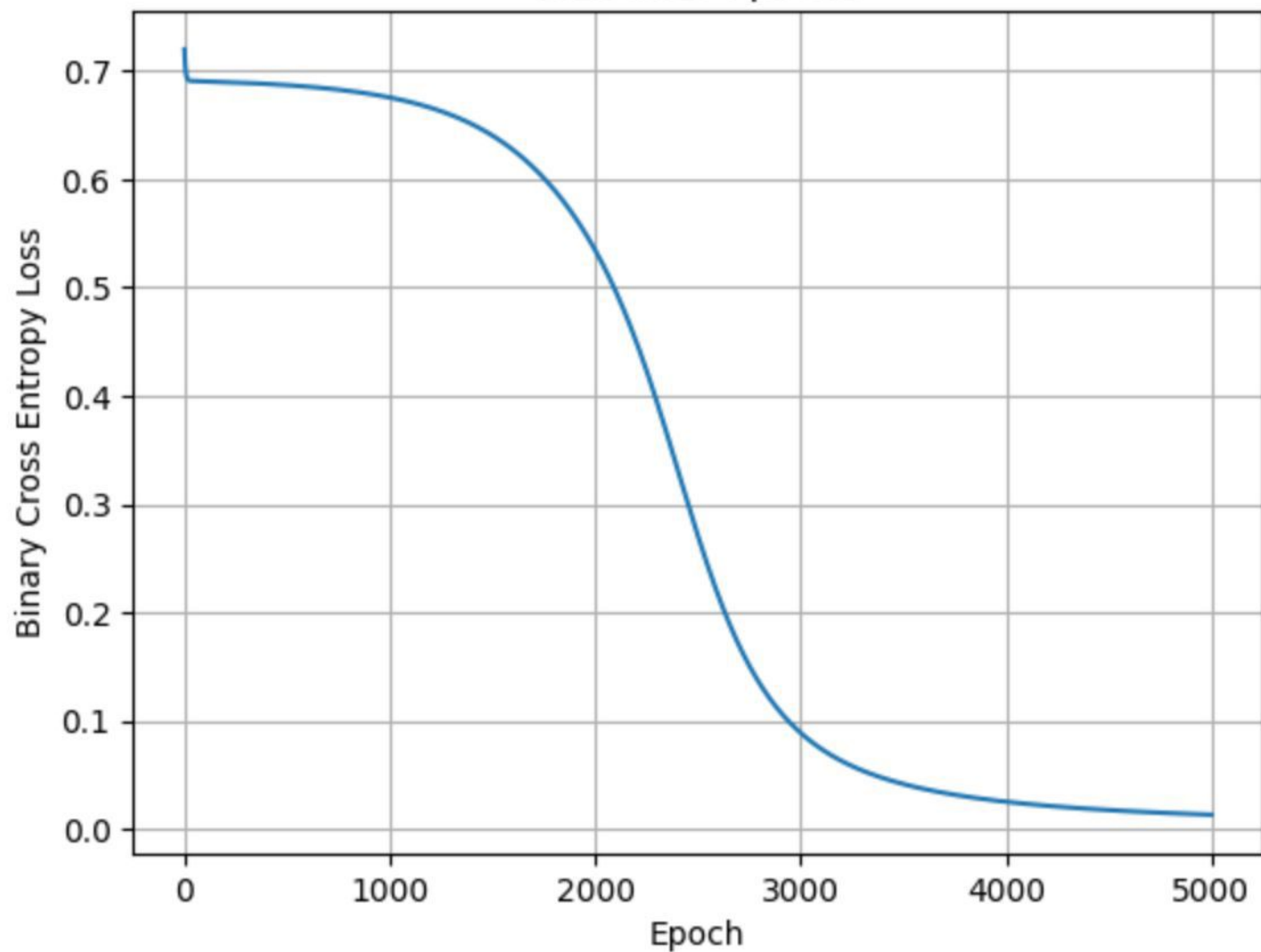
epoch 3500 - loss : 0.0417

epoch 4000 - loss : 0.0253

epoch 4500 - loss : 0.0177



Loss over Epochs





```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
```

```
X = torch.tensor([[0, 0],
                  [0, 1],
                  [1, 0],
                  [1, 1]], dtype=torch.float32)
y = torch.tensor([[0], [1], [1], [0]], dtype=torch.float32)
```

```
torch.manual_seed(42)
input_size = 2
hidden1 = 4
hidden2 = 4
output_size = 1
```

```
W1 = torch.randn(input_size, hidden1, requires_grad=True)
b1 = torch.zeros(hidden1, requires_grad=True)
```

```
W2 = torch.randn(hidden1, hidden2, requires_grad=True)
b2 = torch.zeros(hidden2, requires_grad=True)
```

```
W3 = torch.randn(hidden2, output_size, requires_grad=True)
b3 = torch.zeros(output_size, requires_grad=True)
```

```
lr = 0.1 # learning rate
epochs = 5000
loss_history = []
```